Shizuku CPU 参照表 (RP2040 / RP2350 / ESP32-C6 / ESP32-C5)

状態: **初版**。2026-10-27。対象: `~/github/Shizuku` dev ブランチ (HEAD `1ee7b41` + 未コミット差分)。

1 この文書の使い方

- 第 2〜5 章はチップごとの事実の表で、どのチップも同じ 7 項目を同じ順に並べる (コア / ISA と拡張 / 特権と例外 / 保護 / タイマとカウンタ / メモリ地図 / Shizuku での使用箇所)。
- 値の出どころは 2 種類ある。**repo** はリポジトリのコード (多くは pico-sdk / ESP-IDF の出典を注記済み) から取った値、**DS** は各社のデータシートと Arm / RISC-V の仕様書。初版の推定値は 2026-10-27 に Luna (gpt-6-luna) がデータシートと照合し、確認できたものは印を外した。**[要確認]** が残っているものは一次資料で確認できていない値で、設計判断の根拠にしない。
- 第 6 章は横並びの比較表。第 7 章は「cpu_manager にあるべき処理が各層に散っている」という現状の整理。第 8 章はそれを直した形のサンプルコード (`docs/design/arch/cpu_samples/`)。
- ISA ファミリーとしての設計論は `cortex_m.typ` / `riscv.typ` にある。この文書は個々のチップの事実だけを扱う。

2 RP2040 (Cortex-M0+ × 2)

項目	内容	出典
コア	Cortex-M0+ × 2、対称。最大 133MHz (データシートには条件付きで 200MHz の記述もある)、pico-sdk の既定は 125MHz	RP2040 DS §1.2, §2.15.3。コア数は repo: `chips/rp2040/chip.hpp:13`
ISA と拡張	ARMv6-M (Thumb-1 + BL/MRS/MSR/DMB 等の一部 Thumb-2)。**FPU なし。LDREX/STREX なし** — 原子操作は SIO のハードウェア spinlock (32 本) を IRQ 禁止と組み合わせて作る。除算器は SIO の周辺	repo: `chip.hpp:52`、`arch/armv6m/armv6m.hpp:288–298`
特権と例外	Thread/Handler、CONTROL.nPRIV による特権/非特権。例外フレームは常に 32B (r0-r3, r12, lr, pc, xPSR)。**設定可能なフォールト例外が無い** (HardFault のみ、優先度 −1 固定)。DebugMonitor 例外は無い。NVIC 優先度は上位 2bit (4 段)。外部 IRQ 26 本。VTOR あり。**PSPLIM なし** — スタック下限はソフトで検査する	repo: `chip.hpp:28–36`、`armv6m.hpp:43–68`。RP2040 DS §2.4.2
保護	PMSAv6 MPU、8 region。大きさは 2 の冪 (256B 以上) でその大きさに整列、8 分割の subregion を個別に無効化できる。PRIVDEFENA あり。任意の範囲は張れないので「張れなければ失敗を返す」	repo: `chip.hpp:38–41`、`armv6m.hpp:129–135`
タイマとカウンタ	SysTick 24bit。**DWT CYCCNT なし** (サイクル計測不可)。SoC の TIMER は 64bit µs カウンタ	repo: `armv6m.hpp:44,94–97`
メモリ地図	XIP `0x10000000`、SRAM `0x20000000–0x20042000` (256KiB ストライプ + 4KiB × 2)、APB `0x40000000`、SIO `0xD0000000`、PPB `0xE0000000`	repo: `chip.hpp:15–25`
Shizuku での使用	`arch/armv6m/armv6m.hpp` (arch)、`arch/armv6m/armv6m_ctx.S` (入口)、`boards/rp2040_pico_w/board_rp2040.cpp:155–200` (結線・優先度・MPU)、`modules/pico_sdk_support/rp2040/arch_glue_v6m.cpp` (転送・窓)	

RP2040 固有の注意: flash 書込み中はもう一方のコアを RAM 上で止める (multicore lockout)。その要求は SIO FIFO 割込みで届くので、これを SVC より上 (0x00) に置き、SVC を 0x40 に下げている (`board_rp2040.cpp:163–169`)。4 段しかないため、SVC / SysTick / PendSV の 3 段と SIO FIFO で全部埋まる。

3 RP2350 (Cortex-M33 × 2)

項目	内容	出典
コア	Cortex-M33 × 2 (起動時に Hazard3 RISC-V × 2 と選択可。Shizuku は ARM を使う)。150MHz	計測ログの `clk 150 MHz`。コア数は repo: `chips/rp2350/chip.hpp:11`
ISA と拡張	ARMv8-M Mainline + DSP + FPv5 単精度 FPU (s0-s31) + Security Extension (TrustZone-M)。LDREX/STREX あり。SIO spinlock には erratum (RP2350-E2) があり、pico-sdk は既定でソフトの spinlock を使う	repo: `arch/armv8m/armv8m_ctx.S:6–8`。pico-sdk `spin_lock.h`
特権と例外	CONTROL.nPRIV。MemManage / BusFault / UsageFault / SecureFault / DebugMonitor を個別に持ち、SHCSR で有効化すれば HardFault へ昇格しない。NVIC 優先度は上位 4bit (16 段)。外部 IRQ 52 本。**PSPLIM/MSPLIM あり** (下回ると UsageFault)。例外フレームは 32B、FP を使った文脈では 104B (s0-s15, FPSCR, 予約を追加)。**lazy FP stacking**: FPCCR.ASPEN と LSPEN の両方が 1 のとき、例外入口は FP 領域だけ確保し、ハンドラが FP 命令を使ったときに初めて書く (ASPEN=0 なら自動保存そのものをしない)。CONTROL.FPCA = このスレッドが FP を触ったか (EXC_RETURN bit4 = 0 と対応)	repo: `chip.hpp:24–35`、`armv8m.hpp:90,102–115`。RP2350 DS §3.7.2、Armv8-M ARM B3
保護	PMSAv8 MPU、セキュリティ状態ごとに 8 region。base/limit とも 32B 粒度、limit は含む側のブロック。**region の重なり不可** (広い窓に穴を開けられない)。属性は MAIR0/1 の索引。PRIVDEFENA あり。SAU 8 region	repo: `chip.hpp:37–39`、`armv8m.hpp:317–337,404–423`。RP2350 DS §3.7.2
タイマとカウンタ	SysTick 24bit。DWT CYCCNT 32bit (DEMCR.TRCENA → DWT_CTRL.CYCCNTENA)。SoC の TIMER0/1 は 64bit µs	repo: `armv8m.hpp:181–188,216–230`
メモリ地図	XIP `0x10000000` (XIP キャッシュ 16KiB 2-way)、SRAM `0x20000000–0x20082000` (520KiB)、SIO `0xD0000000`、PPB `0xE0000000`	repo: `chip.hpp:14–20`

項目	内容	出典
Shizuku での使用	`arch/armv8m/armv8m.hpp` (arch)、`armv8m_ctx.S` (入口)、`boards/rp2350_pico2/board.cpp:47–140` (結線・優先度・MPU)、`modules/pico_sdk_support/rp2350/arch_glue.cpp` (転送・窓)	

O3 の測定値について: 3A の測定で O3 だけ O2 の 2 倍前後の値が出た。flash 上のコード配置が XIP キャッシュに収まらない仮説はあるが未検証。ホットパスを SRAM に置く (`docs/design/svc_in_ram.typ`) まで O3 の数字は比較に使わない。

4 ESP32-C6 (RV32IMAC × 1)

項目	内容	出典
コア	HP コア 1 個 (最大 160MHz) + LP コア (RV32IMAC、最大 20MHz)。Shizuku は HP コアだけ使う	C6 DS (CPU and Memory)。コア数は repo: `chips/esp32_c6/chip.hpp:69`
ISA と拡張	RV32IMAC + Zicsr。**FPU なし**。A 拡張 (LR/SC、AMO) あり	repo: `chips/esp32_c6/rv32_c6.hpp:5`
特権と例外	M と U の 2 モード (S なし)。ECALL は mcause 8 (U) / 11 (M)。**trap でレジスタは自動退避されない** (ARM のような例外フレームが無い) — 退避は全部ソフト。割込みは割込み行列で周辺 source を CPU 割込み 1..31 へ写し、PLIC 相当のレジスタで優先度・閾値・許可を付ける。mtvec は vectored (256B 整列の 32 スロット)。trap 中は MIE=0 なので、ECALL の処理中にタイマは入らない (ARM の「SVC 最優先」が構造で成り立つ)	repo: `chips/esp32_c6/trap.cpp:377–407`、`systimer.hpp:7–17`
保護	PMP 16 entry、粒度 4B (TOR/NAPOT)。M モードは L ビットを立てない限り PMP を受けない。U モードは一致 entry が無ければ全部拒否。加えて Espressif 独自の PMA 16 entry (属性・M モードも縛れる)	repo: `chip.hpp:70–73`、`pmp_c6.hpp`、`trap.cpp:56,428–433`
タイマとカウンタ	SYSTIMER 52bit (16MHz) の比較器を oneshot に使う。サイクルカウンタは標準の mcycle ではなく独自 CSR: `mpcer` (0x7E0) / `mpcmr` (0x7E1) / `mpccr` (0x7E2)。U モードからの読出しは未検証	repo: `trap.cpp:1148–1160,1252–1260`、`chip.hpp:32–38`、`systimer.hpp:67–68`
メモリ地図	HP SRAM `0x40800000–0x40880000` (512KiB)、LP SRAM `0x50000000` 16KiB、flash の命令写像 `0x42000000`、周辺 `0x60000000–` (INTMTX `0x60010000`、SYSTIMER `0x6000A000`)	repo: `systimer.hpp:7–10`
Shizuku での使用	`arch/riscv/rv32.hpp` + `rv32_trap.S` (arch)、`chips/esp32_c6/chip.hpp` の `plic_vector_mode`、`chips/esp32_c6/trap.cpp` (1287 行: trap 配送、ecall 表、PMP 切替え、kstack、診断)	

5 ESP32-C5 (RV32IMAC × 1)

項目	内容	出典
コア	HP コア 1 個 (最大 240MHz) + LP コア (最大 48MHz)	C5 DS (CPU and Memory)。コア数は repo: `chips/esp32_c5/chip.hpp:39`
ISA と拡張	RV32IMAC。データシートが明記するのはここまで。Zc 系などの追加拡張の有無は一次資料で確認できていない **[要確認]**	C5 DS
特権と例外	M / U。割込みは **CLIC** (mtvec mode=3 の直接エントリ + mtvt の関数ポインタ表)。Shizuku は全割込みを non-vectored (clicintattr.shv=0) にして 1 入口へ集める。CLIC は mcause に mpp/mpie/mpil を載せるので、例外コードは下位 12bit で比べる	repo: `chip.hpp:13–17`、`arch/riscv/rv32_trap.S` の mcause マスク
保護	PMP 粒度 128B (ESP-IDF の `SOC_CPU_PMP_REGION_GRANULARITY`)。entry 数は 16 と推定 (未確認)。M-only pilot では PMP を張っていない	repo: `chip.hpp:41–47`
タイマとカウンタ	SYSTIMER。サイクルカウンタは未使用 (`HAS_CYCLE_COUNTER = false`)	repo: `chip.hpp:19–21`
メモリ地図	HP SRAM 384KiB、LP SRAM 16KiB	C5 DS
Shizuku での使用	`chips/esp32_c5/` (C6 と同じ形の `clic_direct_mode`)。fail-closed で、`SHIZUKU_C5_SKELETON_PERMITTED` が無いとビルドを拒否する	repo: `rv32_c5.hpp:14–16`

6 横並び比較

項目	RP2040	RP2350	ESP32-C6	ESP32-C5
コア数 (カーネルが使う)	2	2	1	1
最大クロック	133MHz	150MHz	160MHz	240MHz
FPU	なし	単精度 (lazy)	なし	なし
原子命令	なし (SIO spinlock)	LDREX/STREX	LR/SC, AMO	LR/SC, AMO
syscall 命令	SVC	SVC	ECALL	ECALL
HW が積むもの	32B 固定	32B / 104B	なし	なし
遅延切替え	PendSV	PendSV	ソフトの旗 (trap 出口)	同左
優先度	2bit	4bit	PLIC 1–15 **[要確認]**	CLIC

項目	RP2040	RP2350	ESP32-C6	ESP32-C5
個別のフォールト例外	なし	あり	(例外は全部 mtvec)	同左
スタック下限	ソフト	PSPLIM	ソフト	ソフト
保護機構	PMSAv6 × 8	PMSAv8 × 8	PMP × 16 + PMA	PMP × 16? (128B)
保護の粒度	2 の冪 (≥256B)	32B	4B	128B
特権外の既定	PRIVDEFENA	PRIVDEFENA	M は PMP を受けない	同左
サイクルカウンタ	なし	DWT CYCCNT	独自 CSR 0x7E2	未使用

7 現状: cpu_manager にあるべき処理がどこにあるか

今の `source/cpu_manager/init.cpp` は `BOARD::init(BOARD::core_num())` を呼ぶだけで、中身は次に散っている。どれも SDK にも基板にも依存しないので、cpu_manager (または arch) へ寄せられる。

処理	今ある場所	あるべき場所
例外ベクタの登録、SVC > SysTick > PendSV の優先度、SHCSR のフォールト有効化、DebugMon の SHPR 直書き (`0xE000ED18`)	`boards/rp2350_pico2/board.cpp:47–124`、`boards/rp2040_pico_w/board_rp2040.cpp:155–174` (同じことを 2 回)	cpu_manager (順序の規約) + arch (レジスタ)。board は SoC 割込み (USB、SIO FIFO) の優先度だけ
保護の土台 (region0 = コード、残り無効、有効化)	両 board の `protection_init` (ほぼ同文)	cpu_manager。board は範囲を渡すだけ
asm からカーネルへの転送 (`shizuku_svc_dispatch` 等 5 本) と `shizuku_restore_region_window`	`modules/pico_sdk_support/rp2350/arch_glue.cpp:22–94`、`rp2040/arch_glue_v6m.cpp:108–171` (同文 2 本)	カーネル共通 (ARCH で template)。SDK 依存は無い
`unprivileged_floor` / `cycles_per_us` / `time_us`	両 board に同文	board 共通の基底 (pico-sdk 系で 1 本)
「起こされたコアは自分で init を呼ぶ」規約	`rp2350_pico2.hpp` のコメントだけ	cpu_manager の `init_core` / コア起動手順
特権切替え・PMP 切替え・初回起動・タイマ・pend (`plic_vector_mode`)	`chips/esp32_c6/trap.cpp` (1287 行)	特権・PMP・初回起動は arch/riscv + cpu_manager、割込み行列と SYSTIMER は chip
arch 層がチップのマクロを見る (`SHIZUKU_C6_UMODE` 12 か所)	`arch/riscv/rv32.hpp`	chip が型で渡す特性値 + `if constexpr`
カーネル本体がチップ名で分岐 (`SHIZUKU_RP2040`、`__arm__`、`__riscv`)	`source/kernel/dispatch.cpp`、`thread.cpp`	ARCH/CHIP の `constexpr` 特性値
診断の `#if` (BOOT_STAGE_TRACE / SELFTEST_PROGRESS / CRASH_RECORD / BOOT_LOG_WAIT)	`dispatch.cpp` 十数か所、`board_rp2040.cpp` 19、`arch_glue_v6m.cpp` 21	構成が選ぶ DIAG 型 (無効は空 inline)

8 サンプル: 直した形

8.1 方針

1. **カーネルへの入口は 4 本だけ** (`shizuku_svc` / `shizuku_switch` / `shizuku_tick` / `shizuku_fault`)。asm はどれか 1 本を 1 回呼ぶだけにする。今の SVC は 1 回で C を 4 回呼ぶ (`current_context`、`arm_pending_step`、`restore_region_window`、`current_context`)。
2. **SVC/ECALL は切り替えない**。結果を frame に書き (in-place call なら pc / sp / 特権 / 窓も書き換え)、同じスレッドへ戻る。切替えが要れば pend を立てるだけで、PendSV (RISC-V は trap 出口) の 1 か所で切り替える。
3. そのため **SVC 入口は r4-r11 を退避しない**。C が callee-saved を守る。呼び手の callee-saved を消すのは converter の責任 (D1 (c)、2026-10-27 決定) なので、カーネルが消す必要もない。SVC 入口は ARMv6-M でも 4 命令になる。
4. **EXC_RETURN はスレッドのスタックに置かない**。r4-r11 は PendSV がスレッドのスタックへ積むが、非特権スレッドが書き換えられる場所に EXC_RETURN を置くと、ハンドラモードへの復帰を偽造できる。C へ渡し、カーネルのメモリに持つ。
5. 次に走るスレッドの特権・スタック下限・MPU/PMP の窓は、`shizuku_switch` の最後で `cpu_manager::prepare_resume` が 1 回で書く。
6. 優先度の規約 (SVC > タイマ > 切替え) は `static_assert` で縛り、自己テストでレジスタから読み戻して検算する。
7. ISA 差は `HAS_*` の `constexpr` と `if constexpr`、診断は DIAG 型で選ぶ。`#ifdef` を使わない。

未検討の点: (1) in-place call で呼び手と呼び先が別のスタック窓を持つとき、`shizuku_svc` の中で PSP と MPU を書き換える順序。(2) RP2040 の lockout が SVC より上の優先度で入ること (SVC の中で止められても、SVC が文脈を切り替えないので壊れない、と見ているが未確認)。(3) RISC-V の速い ECALL 経路で、呼び手の ra / t* / a2-a7 を消さずに返している点 (converter が clobber 宣言する前提)。サンプルは方向を示すもので、製品コードへの適用は S2 の設計として別に詰める。

サンプルは `docs/design/arch/cpu_samples/` にある。ARMv8-M / ARMv6-M / RV32 の 3 ターゲットで C++ のコンパイル (clang 23、`-std=c++20 -O2 -c`) と asm の組立て (`arm-none-eabi-as`、`clang --target=riscv32`) が通ることだけを確かめた。実機では動かしていない。

8.2 構成

ファイル	役割
`common/kernel_entry.hpp`	カーネルの入口 4 本と規約
`common/cpu_manager.hpp`	1 コアの一生: 起動、優先度、保護の土台、切替え直前の準備
`common/diag.hpp`	診断を型で選ぶ

ファイル	役割
arch/armv8m_entry.S, armv6m_entry.S, rv32_entry.S	例外入口 (SVC / 切替え / フォールト / 初回起動)
arch/armv8m.hpp, armv6m.hpp, rv32.hpp	ISA の操作 (同じ名前で揃える)
chips/rp2350.hpp, rp2040.hpp, esp32_c6.hpp	SoC の数 (コア数、優先度、region 数、メモリ地図)
boards/pico2.hpp	基板の配線と起動コードとの継ぎ目

8.3 ARMv8-M の入口

```
// サンプル: ARMv8-M Mainline (Cortex-M33 / RP2350) の例外入口。全部でこれだけ。
//
// 前提:
//   - スレッドは常に PSP で走る (CONTROL.SPSEL=1)。カーネルは MSP で走る。
//   - SVC はスレッドモードからしか撃たない。よってフレームは常に PSP 上にある。
//   - 切替えは PendSV (最低優先度) だけで起こす。SVC/SysTick/フォールトは pend を立てるだけ。
// 診断用の計測点は置かない。計測は C 側の DIAG 型 (common/diag.hpp) で行う。
.syntax unified
.cpu cortex-m33
.fpu fpv5-sp-d16
.thumb

// ---------------------------------------------------------------------------
// SVC: r0 = 例外フレーム (PSP) を渡して C を 1 回呼ぶだけ。
// r4-r11 は C (AAPCS の callee-saved) が守るので触らない。
// push {r4, lr} は MSP の 8B 整列を保つためと、EXC_RETURN を守るため。
// ---------------------------------------------------------------------------
.thumb_func
.global sample_armv8m_svc_entry
sample_armv8m_svc_entry:
    mrs   r0, psp
    push  {r4, lr}
    bl    shizuku_svc
    pop   {r4, pc}              // pc <- EXC_RETURN: 例外復帰

// ---------------------------------------------------------------------------
// PendSV: ここだけが文脈を切り替える。
//   退避: [FP: s16-s31 (使っていれば)] [r4-r11] を現スレッドの PSP 側へ積む。
//   EXC_RETURN はスタックに置かず C へ渡す (kernel_entry.hpp の注意を参照)。
//   C (shizuku_switch) は CONTROL.nPRIV / PSPLIM / MPU の窓を次のスレッド用に書いてから戻る。
// ---------------------------------------------------------------------------
.thumb_func
.global sample_armv8m_pendsv_entry
sample_armv8m_pendsv_entry:
    mrs   r0, psp
    tst   lr, #0x10             // EXC_RETURN.FType == 0 なら FP 拡張フレーム
    it    eq
    vstmdbeq r0!, {s16-s31}
    stmdb r0!, {r4-r11}
    mov   r1, lr
    bl    shizuku_switch        // r0 = 次の sp, r1 = 次の EXC_RETURN
    ldmia r0!, {r4-r11}
    tst   r1, #0x10
    it    eq
    vldmiaeq r0!, {s16-s31}
    msr   psp, r0
    bx    r1

// ---------------------------------------------------------------------------
// フォールト (MemManage / BusFault / UsageFault / HardFault 共通)。
// 落ちた側がスレッドなら PSP、ハンドラなら MSP にフレームがある (EXC_RETURN.SPSEL)。
// ---------------------------------------------------------------------------
.thumb_func
.global sample_armv8m_fault_entry
sample_armv8m_fault_entry:
    tst   lr, #0x4
    ite   eq
    mrseq r0, msp
    mrsne r0, psp
    mov   r1, lr
    push  {r4, lr}
    bl    shizuku_fault
    pop   {r4, pc}

// SysTick は asm 不要。Cortex-M の例外入口は AAPCS 互換なので、
// ベクタへ C 関数 shizuku_tick を直接置く (arch/armv8m.hpp の install_vectors)。

// ---------------------------------------------------------------------------
// 起動: スレッドスタックへ移って entry を呼ぶ (戻らない)。
//   r0 = スタック先頭, r1 = スタック下限, r2 = entry
// ---------------------------------------------------------------------------
.thumb_func
.global sample_armv8m_enter_thread_mode
sample_armv8m_enter_thread_mode:
    msr   psplim, r1
    msr   psp, r0
```

```
    movs  r3, #2                // SPSEL=1, nPRIV=0
    msr   control, r3
    isb
    blx   r2
1:  b     1b
```

8.4 ARMv6-M の入口

```
// サンプル: ARMv6-M (Cortex-M0+ / RP2040) の例外入口。
// 形は armv8m_entry.S と同じ。違いは Thumb-1 の制約 (stm/ldm は r0-r7 だけ、
// PSPLIM・FP・設定可能なフォールト例外が無い) から来るものだけ。
.syntax unified
.cpu cortex-m0plus
.thumb

.thumb_func
.global sample_armv6m_svc_entry
sample_armv6m_svc_entry:
    mrs   r0, psp
    push  {r4, lr}
    bl    shizuku_svc
    pop   {r4, pc}

// ---------------------------------------------------------------------------
// PendSV: [r4-r7][r8-r11] の 32B を PSP 側へ積む。stmdb が無いので先に sp を下げる。
// ---------------------------------------------------------------------------
.thumb_func
.global sample_armv6m_pendsv_entry
sample_armv6m_pendsv_entry:
    mrs   r0, psp
    subs  r0, #32
    mov   r2, r0
    stmia r2!, {r4-r7}
    mov   r4, r8
    mov   r5, r9
    mov   r6, r10
    mov   r7, r11
    stmia r2!, {r4-r7}
    mov   r1, lr
    bl    shizuku_switch        // r0 = 次の sp, r1 = 次の EXC_RETURN
    adds  r0, #16
    ldmia r0!, {r4-r7}          // 上半分 = r8-r11
    mov   r8, r4
    mov   r9, r5
    mov   r10, r6
    mov   r11, r7
    msr   psp, r0               // sp = 例外フレームの先頭
    subs  r0, #32
    ldmia r0!, {r4-r7}
    bx    r1

// ARMv6-M には HardFault しか無い (優先度 -1 固定)。
.thumb_func
.global sample_armv6m_fault_entry
sample_armv6m_fault_entry:
    mov   r1, lr
    movs  r2, #4
    tst   r1, r2
    bne   1f
    mrs   r0, msp
    b     2f
1:  mrs   r0, psp
2:  push  {r4, lr}
    bl    shizuku_fault
    pop   {r4, pc}

.thumb_func
.global sample_armv6m_enter_thread_mode
sample_armv6m_enter_thread_mode:
    msr   psp, r0               // r1 (下限) は使わない: v6-M に PSPLIM は無い
    movs  r3, #2
    msr   control, r3
    isb
    blx   r2
1:  b     1b
```

8.5 RV32 の入口

```
// サンプル: RV32 (ESP32-C6, M/U モード) の trap 入口。mtvec は direct mode (1 本) を想定。
//
// ARM との違いはハードウェアが何も積まないことだけ。代わりに:
//   - mscratch には常に「今のスレッドの文脈ブロック (カーネルのメモリ)」を指させておく。
//     入口で sp と入れ替えれば、スレッドのスタックに一切触れずに退避先が手に入る。
//     (非特権スレッドのスタックは信用できないので、ここに退避しない)
//   - ECALL の速い経路は a0-a7 をそのまま C の引数にし、a0/a1 をそのまま返す。
//     s0-s11 は C が守る。t*/a2-a7/ra は呼び手の converter が clobber 宣言する。
//   - 切替えは trap の出口で pend を見てから 1 か所で行う (ARM の PendSV に相当)。
.option norvc
```

```
.equ CTX_RA,      0*4       // 文脈ブロック: x1..x31 を x 番号 - 1 の位置に置く
.equ CTX_SP,      1*4
.equ CTX_T0,      4*4
.equ CTX_S0,      7*4
.equ CTX_S1,      8*4
.equ CTX_A0,      9*4
.equ CTX_A1,     10*4
.equ CTX_S2,     17*4
.equ CTX_MEPC,   31*4
.equ CTX_MSTATUS,32*4

.macro SAVE_CALLEE_SAVED      // s0-s11
    sw    s0,  CTX_S0(sp)
    sw    s1,  CTX_S1(sp)
    sw    s2,  (CTX_S2+0*4)(sp)
    sw    s3,  (CTX_S2+1*4)(sp)
    sw    s4,  (CTX_S2+2*4)(sp)
    sw    s5,  (CTX_S2+3*4)(sp)
    sw    s6,  (CTX_S2+4*4)(sp)
    sw    s7,  (CTX_S2+5*4)(sp)
    sw    s8,  (CTX_S2+6*4)(sp)
    sw    s9,  (CTX_S2+7*4)(sp)
    sw    s10, (CTX_S2+8*4)(sp)
    sw    s11, (CTX_S2+9*4)(sp)
.endm

// x1..x31 (sp=x2 を除く) を ctx へ / ctx から。
.macro SAVE_ALL_BUT_SP
    .irp n, 1,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31
    sw    x\n, (\n-1)*4(sp)
    .endr
.endm
.macro LOAD_ALL_BUT_SP
    .irp n, 1,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31
    lw    x\n, (\n-1)*4(sp)
    .endr
.endm

.balign 4
.global sample_rv32_trap_entry
sample_rv32_trap_entry:
    csrrw sp, mscratch, sp      // sp = ctx, mscratch = スレッドの sp
    sw    t0, CTX_T0(sp)
    csrr  t0, mcause
    bltz  t0, .Lslow            // 割込み
    addi  t0, t0, -8            // 8 = U-mode ECALL, 11 = M-mode ECALL
    beqz  t0, .Lecall
    addi  t0, t0, -3
    bnez  t0, .Lslow            // それ以外の例外 (フォールト)

// ---- ECALL の速い経路 ------------------------------------------------
.Lecall:
    csrr  t0, mscratch
    sw    t0, CTX_SP(sp)        // スレッドの sp (in-place call では C が書き換える)
    csrw  mscratch, sp          // mscratch を ctx に戻す
    csrr  t0, mepc
    addi  t0, t0, 4             // ECALL は常に 4 バイト
    sw    t0, CTX_MEPC(sp)
    csrr  t0, mstatus
    sw    t0, CTX_MSTATUS(sp)
    sw    s1, CTX_S1(sp)        // スレッドの s1 を退避してから
    mv    s1, sp                // s1 = ctx として使う (C の呼出しをまたいで残る)
    la    sp, sample_rv32_kernel_stack_top
    mv    a7, s1                // 8 番目の引数 = ctx (frame)。番号と引数は a0-a6
    call  shizuku_svc_rv        // a0/a1 = 結果
    mv    sp, s1
    lw    s1, CTX_S1(sp)
    lw    t0, sample_rv32_pend
    bnez  t0, .Lswitch_after_ecall
    lw    t0, CTX_MEPC(sp)
    csrw  mepc, t0
    lw    t0, CTX_MSTATUS(sp)
    csrw  mstatus, t0
    lw    sp, CTX_SP(sp)
    mret

// ECALL のあとで切替えが要る: 結果 a0/a1 と s0-s11 を ctx へ入れてから切り替える。
.Lswitch_after_ecall:
    sw    a0, CTX_A0(sp)
    sw    a1, CTX_A1(sp)
    SAVE_CALLEE_SAVED
    j     .Lswitch

// ---- 割込み・フォールト: 全レジスタを ctx へ --------------------------------
.Lslow:
    lw    t0, CTX_T0(sp)        // t0 を元に戻してから全部まとめて退避
```

```
    SAVE_ALL_BUT_SP
    csrr  t0, mscratch
    sw    t0, CTX_SP(sp)
    csrw  mscratch, sp
    csrr  t0, mepc
    sw    t0, CTX_MEPC(sp)
    csrr  t0, mstatus
    sw    t0, CTX_MSTATUS(sp)
    mv    s1, sp
    la    sp, sample_rv32_kernel_stack_top
    mv    a0, s1
    csrr  a1, mcause
    call  shizuku_trap_rv       // 割込みの配送 / フォールト処理。pend を立てることがある
    mv    sp, s1
    lw    t0, sample_rv32_pend
    beqz  t0, .Lresume

// ---- 切替え (ARM の PendSV に相当する唯一の場所) ----------------------------
.Lswitch:
    sw    zero, sample_rv32_pend, t0
    mv    a0, sp                // 退避済みの ctx
    li    a1, 0
    la    sp, sample_rv32_kernel_stack_top
    call  shizuku_switch        // a0 = 次の ctx (PMP と mstatus.MPP は C が設定済み)
    mv    sp, a0
    csrw  mscratch, sp

// ---- 全部戻して mret ---------------------------------------------------------
// 初回の起動も同じ道を通る (rv32.hpp の enter_thread_mode が sp = ctx で飛んでくる)。
.global sample_rv32_resume_first
sample_rv32_resume_first:
.Lresume:
    lw    t0, CTX_MEPC(sp)
    csrw  mepc, t0
    lw    t0, CTX_MSTATUS(sp)
    csrw  mstatus, t0
    LOAD_ALL_BUT_SP
    lw    sp, CTX_SP(sp)
    mret
```

8.6 カーネルの入口

```
// サンプル: CPU 層からカーネルへ入る口は 4 本だけ。
//
// どの ISA でも、例外入口の asm はこの 4 本のどれかを 1 回 bl/call するだけにする。
// 今のコードは 1 回の svc で current_context / pending_step / restore_region_window /
// current_context と C を 4 回呼んでいる。窓・特権・スタック上限の設定は、
// 「次に走る文脈が決まった時点」でカーネルの C 側が自分で済ませる。
//
// 規約 (ISA 共通):
//   - SVC/ECALL は文脈を切り替えない。結果を frame に書く (in-place call なら
//     frame の pc/sp/特権と窓も書き換える) だけで、同じスレッドへ戻る。
//   - 切替えが要るときは pend だけ立てる。切替えは最低優先度の 1 か所
//     (ARM は PendSV、RISC-V は trap 出口の pend 確認) でしか起きない。
//   - だから SVC 入口は callee-saved (r4-r11 / s0-s11) を退避しない。C が守る。
//     呼び手の callee-saved を消す責任は caller/callee の converter にある
//     (D1 (c)、2026-10-27 決定)。
#ifndef SHIZUKU_SAMPLE_KERNEL_ENTRY_HPP
#define SHIZUKU_SAMPLE_KERNEL_ENTRY_HPP
#include <stdint.h>

extern "C" {

// SVC / ECALL。frame は ISA の例外フレーム (ARM はハードウェアが積んだもの、
// RISC-V は入口 asm が作った文脈ブロック)。戻り値は無い: 結果は frame に書く。
void shizuku_svc(void *frame);

// 切替え。current の退避が済んだ sp と、そのときの復帰情報 (ARM は EXC_RETURN) を
// 受け取り、次に走る文脈の sp と復帰情報を返す。
// ★復帰情報はスレッドのスタックに置かない。非特権スレッドが書き換えられる場所に
//   EXC_RETURN を置くと、ハンドラモードへの復帰を偽造できる。
// ARM: 戻り値 uint64_t の下位 32bit = sp (r0)、上位 = EXC_RETURN (r1)。
//      (AAPCS では 8 バイトの構造体はメモリ返しになるので、uint64_t で返す)
uint64_t shizuku_switch(uintptr_t saved_sp, uint32_t exc_return);

// 時限タイマの期限。切替えが要るなら pend を立てるだけ。
void shizuku_tick();

// フォールト。落ちたスレッドを止めて pend を立てる。戻り先は PendSV が差し替える。
void shizuku_fault(void *frame, uint32_t exc_return);

} // extern "C"
#endif
```

8.7 cpu_manager

```
// サンプル: cpu_manager = 1 コアの一生 (起動・例外の結線・保護の土台・スレッドモードへの移行・
// 切替え直前の準備) を、arch / chip / board を組み合わせて 1 か所に書く。
```

```
//
// 今のコードでこれに当たる処理は次に散っている:
//   boards/rp2350_pico2/board.cpp:47-124   ベクタ登録・優先度・SHCSR・DebugMon の SHPR 直書き
//   boards/rp2040_pico_w/board_rp2040.cpp:155-174  同じことの v6-M 版
//   boards/*/protection_init                region0 + 残り無効 + 有効化 (2 board でほぼ同文)
//   modules/pico_sdk_support/*/arch_glue*.cpp  dispatch の転送と restore_region_window (同文 2 本)
//   chips/esp32_c6/trap.cpp                 install_traps / interrupt_init / enter_thread_mode
// どれも SDK にも基板にも依存しない。違いは ARCH と CHIP の型が埋める。#ifdef は使わない。
#ifndef SHIZUKU_SAMPLE_CPU_MANAGER_HPP
#define SHIZUKU_SAMPLE_CPU_MANAGER_HPP
#include <stdint.h>

namespace sample {

template <class BOARD, class DIAG> struct cpu_manager {
  using CHIP = typename BOARD::chip;
  using ARCH = typename CHIP::arch;

  // ---- カーネルの不変条件はコンパイル時に縛る ----------------------------------
  // SVC > タイマ > 切替え (ARM は数字が小さいほど高い)。これが 1 コア内の相互排除をただで与える。
  // 移植で静かに崩れやすいので、board の初期化コードではなくここで型として止める。
  // RISC-V は trap 中 MIE=0 で、切替えは trap 出口の 1 か所なので、順序は構造で成り立つ。
  static constexpr bool priority_order_ok() {
    if constexpr (ARCH::HAS_PRIORITY_REGISTERS) {
      using P = typename CHIP::priority;
      return P::SVC < P::TIMER && P::TIMER < P::SWITCH;
    } else {
      return true;
    }
  }
  static_assert(priority_order_ok(), "priority order must be SVC > TIMER > SWITCH");

  // 呼んだコアを起動する。優先度・MPU・SysTick はコアごとに別 (banked) なので、
  // 各コアが自分で呼ぶ。ベクタ表は共有なので登録は core 0 だけ。
  static void init_core(uint32_t core) {
    if (core == 0)
      ARCH::install_vectors();
    if constexpr (ARCH::HAS_PRIORITY_REGISTERS) {
      using P = typename CHIP::priority;
      ARCH::set_priority(ARCH::SVCALL, P::SVC);
      ARCH::set_priority(ARCH::SYSTICK, P::TIMER);
      ARCH::set_priority(ARCH::PENDSV, P::SWITCH);
      if constexpr (ARCH::HAS_FAULT_PRIORITY) {
        ARCH::set_priority(ARCH::MEM_MANAGE, P::FAULT);
        ARCH::set_priority(ARCH::BUS_FAULT, P::FAULT);
        ARCH::set_priority(ARCH::USAGE_FAULT, P::FAULT);
        ARCH::set_priority(ARCH::DEBUG_MONITOR, P::DEBUG);
        ARCH::enable_fault_exceptions();
      }
    } else {
      CHIP::interrupt_init(); // RISC-V: 割込み行列・PLIC/CLIC は chip の仕事
    }
    BOARD::template soc_irq_init<ARCH>(core);
    protection_baseline();
    cycle_counter_enable();
    DIAG::stage(1);
  }

  // 自己テスト用: レジスタから読み戻して順序を確かめる (static_assert は値の検算、
  // こちらは「書けたか」の検算)。
  static bool priorities_ok() {
    if constexpr (ARCH::HAS_PRIORITY_REGISTERS)
      return ARCH::priority(ARCH::SVCALL) < ARCH::priority(ARCH::SYSTICK) &&
             ARCH::priority(ARCH::SYSTICK) < ARCH::priority(ARCH::PENDSV);
    else
      return true;
  }

  // 保護の土台: region0 = ファームのコード (全員 読み+実行)、ほかは全部閉じる。
  // region の外は特権だけ (ARM は PRIVDEFENA、RISC-V は M モードが PMP を受けない)。
  // オブジェクトごとの窓は切替え時 (prepare_resume) に張る。
  static void protection_baseline() {
    if (!ARCH::region_set(0, BOARD::code_begin(), BOARD::code_end(), ARCH::RO_ALL, true))
      DIAG::stage(0xE001); // v6-M の 2 の冪 / PMP の粒度に乗らない。黙って広げない
    for (uint32_t index = 1; index < ARCH::MPU_REGIONS; ++index)
      ARCH::region_disable(index);
    ARCH::protection_enable();
  }

  // 切替えの直前 (shizuku_switch の最後) に 1 回だけ呼ぶ。次に走るスレッドの
  // 特権・スタック下限・窓を、ここでまとめて効かせる。
  // ★今は asm の CTX_RESTORE が pending_step / restore_region_window / current_context を
  //   別々に bl している。次が決まった時点で C が 1 回で済ませれば、asm は値を戻すだけになる。
  template <class THREAD> static void prepare_resume(THREAD &next) {
    ARCH::apply_privilege(next.context, next.privileged);
    if constexpr (ARCH::HAS_STACK_LIMIT)
      ARCH::set_stack_limit(next.stack_limit);
```

```
    for (const auto &window : next.windows) {
      if (window.end == 0)
        ARCH::region_disable(window.region);
      else
        ARCH::region_set(window.region, window.begin, window.end, window.access, false);
    }
  }

  template <class... ARGS> [[noreturn]] static void start(ARGS &...args) {
    ARCH::enter_thread_mode(args...);
  }

private:
  // サイクルカウンタは ISA にある（DWT）こともチップにある（C6 の mpccr）こともある。
  static void cycle_counter_enable() {
    if constexpr (requires { CHIP::cycle_counter_enable(); })
      CHIP::cycle_counter_enable();
    else if constexpr (ARCH::HAS_CYCLE_COUNTER)
      ARCH::cycle_counter_enable();
  }
};

} // namespace sample
#endif
```

8.8 診断

```
// サンプル: 診断を #ifdef ではなく型で選ぶ。
//
// 今のコードは SHIZUKU_BOOT_STAGE_TRACE / SELFTEST_PROGRESS / CRASH_RECORD / BOOT_LOG_WAIT
// を各所の #if で囲んでいる（dispatch.cpp だけで十数か所）。代わりに、構成が DIAG 型を
// 1 つ選び、コードは DIAG::stage(...) と書くだけにする。無効の型は空の inline なので
// 最適化で消え、hot path の命令は増えない。有効の型も全構成でコンパイルされるので、
// 「普段のビルドでは壊れているのに気付かない」が起きない。
#ifndef SHIZUKU_SAMPLE_DIAG_HPP
#define SHIZUKU_SAMPLE_DIAG_HPP
#include <stdint.h>

namespace sample {

// 何もしない（製品構成）。
struct diag_off {
  static constexpr bool ENABLED = false;
  static void stage(uint32_t) {}
  static void fault(const void * /*frame*/, uint32_t /*status*/) {}
};

// リセットをまたいで残る場所（RP なら watchdog scratch）に段階番号を書く。
// 書き込み先は board が知っているので、board の型を受ける。
template <class BOARD> struct diag_stage_trace {
  static constexpr bool ENABLED = true;
  static void stage(uint32_t number) { BOARD::persistent_word(0) = number; }
  static void fault(const void *frame, uint32_t status) {
    BOARD::persistent_word(1) = status;
    BOARD::persistent_word(2) = reinterpret_cast<uintptr_t>(frame);
  }
};

} // namespace sample
#endif
```

8.9 arch: ARMv8-M

```
// サンプル: ARMv8-M Mainline の arch 層。ISA（と Arm が決めたシステム領域）の操作だけを持つ。
// ★ここに無いもの: コア数、優先度の値、MPU region の本数、メモリ地図（chip が持つ）。
//                  どこに何を張るか、クロック、診断出力（board と cpu_manager が持つ）。
// ★SDK を include しない。システム制御空間（0xE000E000-）は Arm の仕様で全チップ共通。
#ifndef SHIZUKU_SAMPLE_ARCH_ARMV8M_HPP
#define SHIZUKU_SAMPLE_ARCH_ARMV8M_HPP
#include <stdint.h>
#include "../common/kernel_entry.hpp"

extern "C" void sample_armv8m_svc_entry();
extern "C" void sample_armv8m_pendsv_entry();
extern "C" void sample_armv8m_fault_entry();
extern "C" [[noreturn]] void sample_armv8m_enter_thread_mode(uintptr_t top, uintptr_t limit,
                                                             void (*entry)());

namespace sample::arch {

template <unsigned MPU_REGIONS_T, unsigned PRIORITY_BITS_T> struct armv8m {
  static constexpr unsigned MPU_REGIONS = MPU_REGIONS_T;
  static constexpr unsigned PRIORITY_BITS = PRIORITY_BITS_T;
  static constexpr uint32_t MPU_GRANULE = 32;
  static constexpr bool HAS_CYCLE_COUNTER = true;
  static constexpr bool HAS_STACK_LIMIT = true;   // PSPLIM
  static constexpr bool HAS_FAULT_PRIORITY = true; // MemManage/BusFault/UsageFault を個別に持つ
  static constexpr bool HAS_PRIORITY_REGISTERS = true;
```

```
// ハードウェアが積む例外フレーム。FP 使用時はこの後ろに s0-s15, FPSCR, 予約が続く（計 104B）。
struct exception_frame_t {
  uint32_t r0, r1, r2, r3, r12, lr, pc, xpsr;
};
// カーネル側に置く文脈は 2 語だけ。r4-r11（と s16-s31）は PendSV がスレッドのスタックへ積む。
struct context_t {
  uintptr_t sp;         // r4-r11 を積んだ後の PSP
  uint32_t exc_return; // スタックに置かない（非特権スレッドに偽造させない）
};

// ---- 例外番号（Arm の定義）----
enum exception : uint32_t {
  HARD_FAULT = 3, MEM_MANAGE = 4, BUS_FAULT = 5, USAGE_FAULT = 6,
  SVCALL = 11, DEBUG_MONITOR = 12, PENDSV = 14, SYSTICK = 15,
};

// ---- ベクタと優先度 ------------------------------------------------------
static volatile uint32_t &reg(uintptr_t address) {
  return *reinterpret_cast<volatile uint32_t *>(address);
}
// RAM 上のベクタ表（VTOR が指す）へカーネルの入口を書く。表を RAM へ置くのは board/起動コードの責任。
static void install_vectors() {
  auto *table = reinterpret_cast<void (**)()>(reg(0xE000ED08)); // VTOR
  table[SVCALL] = sample_armv8m_svc_entry;
  table[PENDSV] = sample_armv8m_pendsv_entry;
  table[SYSTICK] = shizuku_tick; // 例外入口は AAPCS 互換なので C を直接置ける
  table[HARD_FAULT] = sample_armv8m_fault_entry;
  table[MEM_MANAGE] = sample_armv8m_fault_entry;
  table[BUS_FAULT] = sample_armv8m_fault_entry;
  table[USAGE_FAULT] = sample_armv8m_fault_entry;
}
// システム例外 4..15 の優先度は SHPR1-3 にバイト単位で並ぶ（v8-M はバイトアクセス可）。
static void set_priority(exception number, uint8_t priority) {
  reinterpret_cast<volatile uint8_t *>(0xE000ED18)[number - 4] = priority;
}
static uint8_t priority(exception number) {
  return reinterpret_cast<volatile uint8_t *>(0xE000ED18)[number - 4];
}
// 外部割込み（NVIC）の優先度。IPR もバイトアクセス可。
static void set_irq_priority(uint32_t irq, uint8_t priority) {
  reinterpret_cast<volatile uint8_t *>(0xE000E400)[irq] = priority;
}
// MemManage/BusFault/UsageFault を有効化する（無効のままだと HardFault へ昇格し、優先度 -1 になる）。
static void enable_fault_exceptions() {
  reg(0xE000ED24) |= (1u << 16) | (1u << 17) | (1u << 18); // SHCSR
}

// ---- 切替え・タイマ ------------------------------------------------------
static void pend_switch() { reg(0xE000ED04) = 1u << 28; } // ICSR.PENDSVSET
static constexpr uint32_t TIMER_MAX = 0x00FFFFFF;           // SysTick は 24bit
static void timer_oneshot(uint32_t cycles) {
  reg(0xE000E010) = 0;          // CSR: 止める
  reg(0xE000E014) = cycles - 1; // RVR
  reg(0xE000E018) = 0;          // CVR: 書くと 0 になり、次の刻みで RVR を読み込む
  reg(0xE000E010) = 0b111;      // CLKSOURCE=CPU, TICKINT, ENABLE
}
static void timer_cancel() { reg(0xE000E010) = 0; }

// ---- サイクルカウンタ（DWT）---------------------------------------------
static bool cycle_counter_enable() {
  reg(0xE000EDFC) |= 1u << 24; // DEMCR.TRCENA
  reg(0xE0001000) |= 1u;       // DWT_CTRL.CYCCNTENA
  return (reg(0xE0001000) & 1u) != 0;
}
static uint32_t cycle_counter() { return reg(0xE0001004); }

// ---- 特権と FP の状態（今走っているコードの自己申告）--------------------------
static uint32_t control() {
  uint32_t value;
  asm volatile("mrs %0, control" : "=r"(value));
  return value;
}
static bool privileged() { return (control() & 1u) == 0; }
// CONTROL.FPCA: このスレッドが FP を触ったか。converter の FP 消去はこれで判定する
// （コンパイルオプションではなく実行時に読む。2026-10-27 決定）。
static bool fp_active() { return (control() & 4u) != 0; }
// 例外の中から「戻り先の特権」を変える。復帰した時点で効く。
// ARM では特権は CPU のレジスタ（CONTROL.nPRIV）なので、文脈ではなくレジスタを書く。
static void apply_privilege(context_t &, bool privileged) {
  uint32_t value = control();
  value = privileged ? (value & ~1u) : (value | 1u);
  asm volatile("msr control, %0\n isb" ::"r"(value) : "memory");
}
static void set_stack_limit(uintptr_t limit) {
  asm volatile("msr psplim, %0" ::"r"(limit));
}
```

```
  // ---- MPU (PMSAv8) ------------------------------------------------------
  // region は重ねられない。base/limit は 32B 粒度で limit は「含む側の最後のブロック」。
  enum access : uint32_t { RW_PRIV = 0b00, RW_ALL = 0b01, RO_PRIV = 0b10, RO_ALL = 0b11 };
  // 32B 粒度なので前後に最大 31B はみ出す。はみ出しを許さない配置は呼ぶ側（割当て）の責任。
  // armv6m と同じ形にするため bool を返す（v8-M は常に張れる）。
  static bool region_set(uint32_t index, uintptr_t base, uintptr_t end_exclusive, access ap,
                         bool execute) {
    reg(0xE000ED98) = index;                                          // RNR
    reg(0xE000ED9C) = (base & ~31u) | (ap << 1) | (execute ? 0u : 1u);        // RBAR
    reg(0xE000EDA0) = ((end_exclusive - 1) & ~31u) | 1u;                      // RLAR (attr0)
    return true;
  }
  static void region_disable(uint32_t index) {
    reg(0xE000ED98) = index;
    reg(0xE000EDA0) = 0;
  }
  // PRIVDEFENA: region の外は特権だけ。単一アドレス空間の「カーネル空間」はこれで作る。
  static void protection_enable() {
    reg(0xE000EDC0) = 0xFF;          // MAIR0: attr0 = 通常メモリ (WB)
    reg(0xE000ED94) = 0b101;         // MPU_CTRL: PRIVDEFENA | ENABLE
    asm volatile("dsb\n isb" ::: "memory");
  }

  [[noreturn]] static void enter_thread_mode(uintptr_t top, uintptr_t limit, void (*entry)()) {
    sample_armv8m_enter_thread_mode(top, limit, entry);
  }
};

} // namespace sample::arch
#endif
```

8.10 arch: ARMv6-M

```
// サンプル: ARMv6-M の arch 層。armv8m.hpp と同じ名前の操作を、v6-M でできる形で持つ。
// 無いものは HAS_* を false にし、空の操作を置く。呼ぶ側（cpu_manager）は
// if constexpr で分けるので #ifdef は要らない。
#ifndef SHIZUKU_SAMPLE_ARCH_ARMV6M_HPP
#define SHIZUKU_SAMPLE_ARCH_ARMV6M_HPP
#include <stdint.h>
#include "../common/kernel_entry.hpp"

extern "C" void sample_armv6m_svc_entry();
extern "C" void sample_armv6m_pendsv_entry();
extern "C" void sample_armv6m_fault_entry();
extern "C" [[noreturn]] void sample_armv6m_enter_thread_mode(uintptr_t top, uintptr_t limit,
                                                             void (*entry)());

namespace sample::arch {

template <unsigned MPU_REGIONS_T, unsigned PRIORITY_BITS_T> struct armv6m {
  static constexpr unsigned MPU_REGIONS = MPU_REGIONS_T;
  static constexpr unsigned PRIORITY_BITS = PRIORITY_BITS_T;
  static constexpr bool HAS_CYCLE_COUNTER = false;  // DWT CYCCNT が無い
  static constexpr bool HAS_STACK_LIMIT = false;    // PSPLIM が無い。下限はソフトで検査する
  static constexpr bool HAS_FAULT_PRIORITY = false; // HardFault だけ（優先度 -1 固定）
  static constexpr bool HAS_PRIORITY_REGISTERS = true;

  struct exception_frame_t {
    uint32_t r0, r1, r2, r3, r12, lr, pc, xpsr;
  };
  struct context_t {
    uintptr_t sp;
    uint32_t exc_return;
  };
  enum exception : uint32_t { HARD_FAULT = 3, SVCALL = 11, PENDSV = 14, SYSTICK = 15 };

  static volatile uint32_t &reg(uintptr_t address) {
    return *reinterpret_cast<volatile uint32_t *>(address);
  }
  static void install_vectors() {
    auto *table = reinterpret_cast<void (**)()>(reg(0xE000ED08));
    table[SVCALL] = sample_armv6m_svc_entry;
    table[PENDSV] = sample_armv6m_pendsv_entry;
    table[SYSTICK] = shizuku_tick;
    table[HARD_FAULT] = sample_armv6m_fault_entry;
  }
  // v6-M の SHPR2/3 はワードアクセスのみ。該当バイトだけを書き換える。
  //   SVCall = SHPR2[31:24], PendSV = SHPR3[23:16], SysTick = SHPR3[31:24]
  static uint32_t shpr_address(exception number) { return number == SVCALL ? 0xE000ED1C : 0xE000ED20; }
  static uint32_t shpr_shift(exception number) { return 8u * (number % 4u); }
  static void set_priority(exception number, uint8_t priority) {
    auto &shpr = reg(shpr_address(number));
    const uint32_t shift = shpr_shift(number);
    shpr = (shpr & ~(0xFFu << shift)) | (uint32_t(priority) << shift);
  }
  static uint8_t priority(exception number) {
    return uint8_t(reg(shpr_address(number)) >> shpr_shift(number));
```

```
  }
  static void set_irq_priority(uint32_t irq, uint8_t priority) {
    auto &ipr = reg(0xE000E400 + 4u * (irq / 4u));
    const uint32_t shift = 8u * (irq % 4u);
    ipr = (ipr & ~(0xFFu << shift)) | (uint32_t(priority) << shift);
  }
  static void enable_fault_exceptions() {}

  static void pend_switch() { reg(0xE000ED04) = 1u << 28; }
  static constexpr uint32_t TIMER_MAX = 0x00FFFFFF;
  static void timer_oneshot(uint32_t cycles) {
    reg(0xE000E010) = 0;
    reg(0xE000E014) = cycles - 1;
    reg(0xE000E018) = 0;
    reg(0xE000E010) = 0b111;
  }
  static void timer_cancel() { reg(0xE000E010) = 0; }

  static bool cycle_counter_enable() { return false; }
  static uint32_t cycle_counter() { return 0; }

  static uint32_t control() {
    uint32_t value;
    asm volatile("mrs %0, control" : "=r"(value));
    return value;
  }
  static bool privileged() { return (control() & 1u) == 0; }
  static bool fp_active() { return false; } // FPU が無い
  static void apply_privilege(context_t &, bool privileged) {
    uint32_t value = control();
    value = privileged ? (value & ~1u) : (value | 1u);
    asm volatile("msr control, %0\n isb" ::"r"(value) : "memory");
  }
  static void set_stack_limit(uintptr_t) {}

  // ---- MPU (PMSAv6) ------------------------------------------------------
  // region の大きさは 2 の冪（256B 以上）で、base はその大きさに整列する。8 分割の
  // subregion を無効化して穴を作れる。張れない範囲は false を返す（黙って広げない）。
  enum access : uint32_t { RW_PRIV = 0b001, RW_ALL = 0b011, RO_PRIV = 0b101, RO_ALL = 0b110 };
  static bool region_set(uint32_t index, uintptr_t base, uintptr_t end_exclusive, access ap,
                         bool execute) {
    const uintptr_t size = end_exclusive - base;
    if (size < 256 || (size & (size - 1)) != 0 || (base & (size - 1)) != 0)
      return false;
    const uint32_t size_field = 31u - uint32_t(__builtin_clz(size)) - 1u; // size = 2^(SIZE+1)
    reg(0xE000ED98) = index;                                              // RNR
    reg(0xE000ED9C) = base;                                               // RBAR
    reg(0xE000EDA0) = (execute ? 0u : 1u << 28) | (ap << 24) | (size_field << 1) | 1u; // RASR
    return true;
  }
  static void region_disable(uint32_t index) {
    reg(0xE000ED98) = index;
    reg(0xE000EDA0) = 0;
  }
  static void protection_enable() {
    reg(0xE000ED94) = 0b101; // PRIVDEFENA | ENABLE
    asm volatile("dsb\n isb" ::: "memory");
  }

  [[noreturn]] static void enter_thread_mode(uintptr_t top, uintptr_t limit, void (*entry)()) {
    sample_armv6m_enter_thread_mode(top, limit, entry);
  }
};

} // namespace sample::arch
#endif
```

8.11 arch: RV32

```
// サンプル: RV32（M/U モード + PMP）の arch 層。RISC-V の特権仕様で決まる操作だけを持つ。
// 名前は armv8m.hpp / armv6m.hpp とそろえてある（cpu_manager が同じ書き方で呼べるように）。
// ★割込みコントローラ（C6 の割込み行列 + PLIC、C5 の CLIC）とタイマ（SYSTIMER）は
//   RISC-V の仕様の外なので chip が持つ。ARM で NVIC/SysTick が arch にあるのと非対称に
//   見えるが、境界は「ISA の仕様書に書いてあるか」で引く。
#ifndef SHIZUKU_SAMPLE_ARCH_RV32_HPP
#define SHIZUKU_SAMPLE_ARCH_RV32_HPP
#include <stdint.h>

extern "C" void sample_rv32_trap_entry();
extern "C" volatile uint32_t sample_rv32_pend;

namespace sample::arch {

// PMP_ENTRIES / PMP_GRANULE は chip が渡す（C6 は 16 本・4B、C5 は 128B）。
template <unsigned PMP_ENTRIES_T, unsigned PMP_GRANULE_T> struct rv32 {
  static constexpr unsigned PMP_ENTRIES = PMP_ENTRIES_T;
  static constexpr unsigned PMP_GRANULE = PMP_GRANULE_T;
  // region 1 個 = TOR の entry 2 本（下端と上端）。
```

```
  static constexpr unsigned MPU_REGIONS = PMP_ENTRIES / 2;
  static constexpr bool HAS_PRIORITY_REGISTERS = false; // 例外の優先度は CPU に無い（PLIC は chip）
  static constexpr bool HAS_FAULT_PRIORITY = false;
  static constexpr bool HAS_STACK_LIMIT = false;
  static constexpr bool HAS_CYCLE_COUNTER = false;      // 標準の mcycle は C6 に無い。chip の CSR を使う

  // 文脈ブロック（rv32_entry.S の CTX_* と同じ並び）。カーネルのメモリに置く。
  struct context_t {
    uint32_t x[31]; // x1..x31 (x[1] = sp)
    uint32_t mepc;
    uint32_t mstatus;
  };
  static_assert(sizeof(context_t) == 33 * 4);

  static constexpr uint32_t MSTATUS_MPIE = 1u << 7;
  static constexpr uint32_t MSTATUS_MPP = 3u << 11; // 11 = M, 00 = U

  // direct mode（下位 2bit = 0）。ベクタ化は chip の選択（C6 の PLIC vector mode など）。
  static void install_vectors() {
    asm volatile("csrw mtvec, %0" ::"r"(reinterpret_cast<uintptr_t>(&sample_rv32_trap_entry))
                 : "memory");
  }
  // 「切替え要求」はソフトの旗。trap 出口が見る（ARM の PENDSVSET に相当）。
  static void pend_switch() { sample_rv32_pend = 1; }

  // 戻り先の特権は mstatus.MPP。ARM と違って CPU のレジスタではなく文脈の値なので、
  // 文脈を書き換えるだけで、trap 出口の mret が効かせる。
  static void apply_privilege(context_t &context, bool privileged) {
    context.mstatus = (context.mstatus & ~MSTATUS_MPP) | (privileged ? MSTATUS_MPP : 0u);
    context.mstatus |= MSTATUS_MPIE; // スレッドは割込み許可で走る
  }
  static void set_stack_limit(uintptr_t) {}

  // ---- PMP (TOR) ------------------------------------------------------------
  // cfg 1 本 = 8bit: L(7) A(4:3) X(2) W(1) R(0)。A = 1 が TOR。
  // *M モードは L を立てない限り PMP の制限を受けない（Arm の PRIVDEFENA に相当する
  //   「特権は全部見える」が既定）。U モードは一致する entry が無いアクセスを全部拒否する。
  enum access : uint8_t { RO_ALL = 0b001, RW_ALL = 0b011 };
  static bool region_set(uint32_t index, uintptr_t base, uintptr_t end_exclusive, access ap,
                         bool execute) {
    if ((base | end_exclusive) & (PMP_GRANULE - 1))
      return false; // 粒度に乗らない範囲は張らない（黙って広げない）
    const unsigned low = 2 * index, high = 2 * index + 1;
    write_pmpaddr(low, uint32_t(base >> 2));
    write_pmpaddr(high, uint32_t(end_exclusive >> 2));
    write_pmpcfg(low, 0); // 下端用は OFF
    write_pmpcfg(high, uint8_t(ap | (execute ? 0b100u : 0u) | (1u << 3)));
    return true;
  }
  static void region_disable(uint32_t index) { write_pmpcfg(2 * index + 1, 0); }
  static void protection_enable() {} // PMP は常に有効。U モードは既定で全部拒否

  static void write_pmpaddr(unsigned index, uint32_t value); // csrw pmpaddrN（N は即値なので表で引く）
  static void write_pmpcfg(unsigned index, uint8_t cfg);     // pmpcfg(N/4) の該当バイトを書き換える

  // 初回もふつうの trap 出口（全部戻して mret）と同じ道で入る。
  [[noreturn]] static void enter_thread_mode(context_t &context) {
    asm volatile("csrw mscratch, %0\n mv sp, %0\n j sample_rv32_resume_first" ::"r"(&context));
    __builtin_unreachable();
  }
};

} // namespace sample::arch
#endif
```

8.12 chip と board

```
// サンプル: chip = SoC の「数」だけ。関数はコア番号の読み出しのような 1 行のものに限る。
// 値の出典は pico-sdk と RP2350 データシート（cpu_reference.typ の表を参照）。
#ifndef SHIZUKU_SAMPLE_CHIP_RP2350_HPP
#define SHIZUKU_SAMPLE_CHIP_RP2350_HPP
#include <stdint.h>
#include "../arch/armv8m.hpp"

namespace sample::chips {

struct rp2350 {
  using arch = sample::arch::armv8m</*MPU_REGIONS*/ 8, /*PRIORITY_BITS*/ 4>;
  static constexpr uint32_t CORE_COUNT = 2;

  // ---- メモリ地図 ----
  static constexpr uintptr_t XIP_BASE = 0x10000000;
  static constexpr uintptr_t SRAM_BASE = 0x20000000;
  static constexpr uintptr_t SRAM_END = 0x20082000; // 520KiB

  // ---- 優先度（0 が最も高い。上位 4bit だけ効く）----
  // カーネルの不変条件 SVC > タイマ > 切替え は cpu_manager が static_assert する。
  // chip が決めるのは、その間にどの SoC 割込みを挟むかだけ。
```

```
  struct priority {
    static constexpr uint8_t SVC = 0x00;
    static constexpr uint8_t TIMER = 0x40;
    static constexpr uint8_t FAULT = 0x80;     // USB (0x20) より下: 報告中も USB が回る
    static constexpr uint8_t DEBUG = 0xA0;
    static constexpr uint8_t SWITCH = 0xFF;    // PendSV は最低
  };

  // ---- SoC 割込み ----
  static constexpr uint32_t IRQ_USBCTRL = 14;
  static constexpr uint32_t IRQ_SIO_FIFO_CORE0 = 25; // multicore lockout (flash 書込み中の停止)

  static uint32_t core_id() { return *reinterpret_cast<volatile uint32_t *>(0xD0000000); }
};

} // namespace sample::chips
#endif
```

```
// サンプル: RP2040 の chip。形は rp2350.hpp と同じ。
#ifndef SHIZUKU_SAMPLE_CHIP_RP2040_HPP
#define SHIZUKU_SAMPLE_CHIP_RP2040_HPP
#include <stdint.h>
#include "../arch/armv6m.hpp"

namespace sample::chips {

struct rp2040 {
  using arch = sample::arch::armv6m</*MPU_REGIONS*/ 8, /*PRIORITY_BITS*/ 2>;
  static constexpr uint32_t CORE_COUNT = 2;

  static constexpr uintptr_t XIP_BASE = 0x10000000;
  static constexpr uintptr_t SRAM_BASE = 0x20000000;
  static constexpr uintptr_t SRAM_END = 0x20042000; // 264KiB

  // 優先度は 4 段 (0x00/0x40/0x80/0xC0) しかない。
  // *SIO FIFO (multicore lockout) を SVC より上に置く。flash を書く側のコアが、
  //   SVC の中にいる相手を止められるようにするため。よって SVC は 0x40。
  struct priority {
    static constexpr uint8_t SIO_FIFO = 0x00;
    static constexpr uint8_t SVC = 0x40;
    static constexpr uint8_t TIMER = 0x80;
    static constexpr uint8_t USB = 0x80;
    static constexpr uint8_t SWITCH = 0xC0;
  };

  static constexpr uint32_t IRQ_USBCTRL = 5;
  static constexpr uint32_t IRQ_SIO_PROC0 = 15;

  static uint32_t core_id() { return *reinterpret_cast<volatile uint32_t *>(0xD0000000); }
};

} // namespace sample::chips
#endif
```

```
// サンプル: ESP32-C6 の chip。RISC-V の仕様の外にあるもの (割込み行列、PLIC、SYSTIMER) はここ。
#ifndef SHIZUKU_SAMPLE_CHIP_ESP32_C6_HPP
#define SHIZUKU_SAMPLE_CHIP_ESP32_C6_HPP
#include <stdint.h>
#include "../arch/rv32.hpp"

namespace sample::chips {

struct esp32_c6 {
  using arch = sample::arch::rv32</*PMP_ENTRIES*/ 16, /*PMP_GRANULE*/ 4>;
  static constexpr uint32_t CORE_COUNT = 1; // HP コア。LP コアはカーネルの外

  static constexpr uintptr_t HP_SRAM_BASE = 0x40800000;
  static constexpr uintptr_t HP_SRAM_END = 0x40880000; // 512KiB

  // ---- 割込み: 周辺の source を割込み行列で CPU 割込み 1..31 へ写し、PLIC で優先度を付ける ----
  static constexpr uintptr_t INTMTX_BASE = 0x60010000;
  static constexpr uintptr_t SYSTIMER_BASE = 0x6000A000;
  static constexpr uint32_t SOURCE_SYSTIMER_TARGET0 = 57;
  static constexpr uint32_t CPU_IRQ_TIMER = 17; // chips/esp32_c6/systimer.hpp:21 と同じ

  struct priority {
    // RISC-V では ECALL は例外 (同期) なので優先度を持たない。trap 中は MIE=0 なので
    // 「SVC 最優先」は構造で成り立つ。タイマだけ PLIC の優先度を付ける。
    static constexpr uint8_t TIMER = 3;     // PLIC は priority > threshold で通す
    static constexpr uint8_t THRESHOLD = 1; // IDF の RVHAL_INTR_ENABLE_THRESH と同じ
  };

  // SYSTIMER の 1 本を oneshot に使う (16MHz の 52bit カウンタ)。実装は chip の .cpp。
  static void timer_oneshot(uint32_t ticks);
  static void timer_cancel();
  static void interrupt_init(); // 行列の写像・PLIC の優先度と閾値・mie

  // マシンの拡張 CSR: mpcer(0x7E0) / mpcmr(0x7E1) / mpccr(0x7E2) でサイクル数を数える。
```

```
  static bool cycle_counter_enable() {
    asm volatile("csrw 0x7E0, %0\n csrw 0x7E1, %0" ::"r"(1u));
    return true;
  }
  static uint32_t cycle_counter() {
    uint32_t value;
    asm volatile("csrr %0, 0x7E2" : "=r"(value));
    return value;
  }
};

} // namespace sample::chips
#endif
```

```
// サンプル: board = 基板の配線と、起動コード・SDK との継ぎ目だけ。
// 例外の結線・優先度の規約・MPU の基本配置は持たない（cpu_manager が持つ）。
#ifndef SHIZUKU_SAMPLE_BOARD_PICO2_HPP
#define SHIZUKU_SAMPLE_BOARD_PICO2_HPP
#include <stdint.h>
#include "../chips/rp2350.hpp"

extern "C" char __flash_binary_end; // リンカが置く。ファーム本体の終端
extern "C" char __end__;            // 静的領域の終端 = ヒープ先頭

namespace sample::boards {

struct pico2 {
  using chip = sample::chips::rp2350;

  // ファームのコードの範囲（region0 = 読み+実行に張る範囲）。
  static uintptr_t code_begin() { return chip::XIP_BASE; }
  static uintptr_t code_end() { return reinterpret_cast<uintptr_t>(&__flash_binary_end); }
  // 非特権から届く最下位アドレス。これより下はカーネルの静的領域。
  static uintptr_t unprivileged_floor() { return (reinterpret_cast<uintptr_t>(&__end__) + 31) & ~31u; }

  // この基板で使う SoC 割込みの優先度。カーネルの 3 段（SVC/タイマ/切替え）には触らない。
  template <class ARCH> static void soc_irq_init(uint32_t core) {
    ARCH::set_irq_priority(chip::IRQ_USBCTRL, 0x20); // 落ちた報告を USB で出し切るため FAULT より上
    ARCH::set_irq_priority(chip::IRQ_SIO_FIFO_CORE0 + core, 0x00);
  }

  static uint32_t clock_hz();                    // 実クロック（SDK の clock_get_hz を呼ぶ .cpp）
  static void launch_core(void (*entry)());      // もう一方のコアを起こす（SDK の multicore）
  static void diag_write(const char *text, uint32_t length);
  static volatile uint32_t &persistent_word(uint32_t index) {
    return reinterpret_cast<volatile uint32_t *>(0x400D800C)[index]; // watchdog SCRATCH0..
  }
};

} // namespace sample::boards
#endif
```